Day 4

Smart Traffic System Workshop

Webcam + Drawing on Frames

VideoCapture | Live feed loop | Drawing shapes | Text on frame | waitKey | Mini project

Day recap

- cv2.imread() — image file load karna
- cv2.imshow() — image ya frame screen pe dikhana
- img.shape — (height, width, channels) properties
- BGR format — OpenCV mein Blue pehle aata hai, Red baad mein
- cv2.resize() — image chhoti ya badi karna

Topic 1 — Webcam se Live Video Lena (VideoCapture)

Real life analogy

Sochlo ek security guard hai jo CCTV monitor dekhta rehta hai — continuously, ek ek frame.

VideoCapture(0) = camera kholo (0 = pehla camera / built-in webcam)

cap.read() = ek frame lo camera se

Yeh loop mein chalta hai — jab tak 'q' key na dabao, frames aate rehte hain.

Traffic system mein: yahi loop har frame mein YOLO detection chalayega.

Webcam ka simplest code:

```
import cv2


# Camera kholo (0 = default camera)

cap = cv2.VideoCapture(0)


# Camera properly khula ya nahi?

if not cap.isOpened():

    print('Camera nahi mila!')

else:

    print('Camera ready hai!')


# Ek frame lo aur dikhao

ret, frame = cap.read()   # ret = True/False, frame = image

if ret:

    cv2.imshow('My Camera', frame)
```

```
    cv2.waitKey(0)


cap.release()             # Camera band karo

cv2.destroyAllWindows()
```

Live loop — har frame dikhao:

```
import cv2


cap = cv2.VideoCapture(0)


while True:                          # Hamesha chalta raho

    ret, frame = cap.read()          # Naya frame lo

    if not ret:                      # Frame nahi mila?

        print('Frame nahi aaya!')

        break


    cv2.imshow('Live Camera', frame) # Frame dikhao

    key = cv2.waitKey(1)             # 1ms wait karo

    if key == ord('q'):              # 'q' dabate hi band karo

        break


cap.release()                        # Camera band karo

cv2.destroyAllWindows()
```

Output / Terminal:

```
(Live webcam window khulegi)

(Camera feed aata rahega)

('q' key dabao — window band ho jayegi)
```

Code	Kya karta hai?
`cap = VideoCapture(0)`	Laptop webcam (index 0)
`cap = VideoCapture(1)`	Baahri / USB camera (index 1)
`cap = VideoCapture(2)`	Teesra camera — Jetson pe koi bhi lane
`cap.isOpened()`	Camera mila ya nahi — True/False
`ret, frame = cap.read()`	ret = success, frame = image array
`cap.release()`	Camera properly band karna — zaroor likhna!

Try it yourself:

1. Upar wala live loop code type karo.
2. Run karo — camera window dikhi?
3. 'q' dabao — band hua?
4. VideoCapture(0) se (1) karo — kya hota hai?

Topic 2 — Frame pe Shapes Draw karna

Real life analogy

Traffic system mein YOLO ek gaadi detect karta hai — uske around ek box draw karna padta hai.

Yahi bounding box hota hai — cv2.rectangle() se draw hota hai.

Colour hamesha BGR format mein — Red = (0,0,255), Green = (0,255,0), Blue = (255,0,0).

Rectangle, Circle, Line — teeno ek saath:

```
import cv2

import numpy as np


# Blank image banana (kale background pe draw karenge)

canvas = np.zeros((480, 640, 3), dtype='uint8')


# Rectangle — bounding box jaise

# cv2.rectangle(image, top-left, bottom-right, color_BGR, thickness)

cv2.rectangle(canvas, (50, 50), (250, 200), (0, 255, 0), 2)   # Green box


# Circle — detection center mark karne ke liye

# cv2.circle(image, center, radius, color_BGR, thickness)

cv2.circle(canvas, (400, 150), 60, (0, 0, 255), 3)            # Red circle


# Line — lane boundary dikhane ke liye

# cv2.line(image, start_point, end_point, color_BGR, thickness)

cv2.line(canvas, (0, 300), (640, 300), (255, 255, 0), 2)      # Yellow line


cv2.imshow('Drawing', canvas)

cv2.waitKey(0)

cv2.destroyAllWindows()
```

BGR Color Tuple	Traffic System mein use
`(0, 255, 0)`	Green — vehicle detected
`(0, 0, 255)`	Red — stop signal / alert
`(255, 255, 0)`	Cyan — lane boundary
`(0, 255, 255)`	Yellow — warning
`(255, 255, 255)`	White — text background
`(0, 0, 0)`	Black — blank canvas

Try it yourself:

1. Blank canvas pe teen rectangles draw karo — teen alag colors.
2. Ek circle beech mein draw karo.
3. Ek horizontal line canvas ke aadhe mein draw karo.

Topic 3 — Frame pe Text Likhna (putText)

Real life analogy

Traffic system mein detected vehicles ka count screen pe dikhana hota hai.

Jaise CCTV monitor pe 'North: 12 vehicles' likhta dikhta hai —

cv2.putText() se hum yahi karte hain: live frame pe text overlay karte hain.

putText syntax — parameters samjho:

```
import cv2

import numpy as np


canvas = np.zeros((480, 640, 3), dtype='uint8')


# cv2.putText(image, text, position, font, scale, color, thickness)

cv2.putText(

    canvas,

    'North Lane: 12 Vehicles',      # Text

    (30, 60),                        # Position (x, y) — top-left se

    cv2.FONT_HERSHEY_SIMPLEX,        # Font style

    0.8,                             # Font scale (size)

    (0, 255, 0),                     # Color — Green (BGR)

    2                                # Thickness

)
```

```
# Multiple lines — alag alag y position
lanes = [('North', 12), ('South', 5), ('East', 20), ('West', 8)]
for i, (lane, count) in enumerate(lanes):
    y_pos = 120 + i * 50            # Har line 50px neeche
    text = f'{lane}: {count} vehicles'
    cv2.putText(canvas, text, (30, y_pos),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

cv2.imshow('Lane Counter', canvas)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Font Style	Kab use karein?
FONT_HERSHEY_SIMPLEX	Sabse common — clean aur simple
FONT_HERSHEY_DUPLEX	Thoda bold — headings ke liye
FONT_HERSHEY_COMPLEX	Italic style
FONT_HERSHEY_PLAIN	Chhota aur simple

Try it yourself:

1. Blank canvas pe apna naam likhne wala code banao.
2. 4 lines mein 4 alag lanes aur counts likhne ki koshish karo.
3. Ek line ka color green, doosri red karo.

Topic 4 — waitKey aur Keyboard Controls

Real life analogy

waitKey = system ko thoda rukne dena aur key check karna.

Jaise security guard har 1 second mein check karta hai — koi alarm button daba to nahi?

waitKey(1) = 1ms wait karo aur key check karo (live video ke liye)

waitKey(0) = koi bhi key dabaane tak rokho (image display ke liye)

Multiple keys handle karna:

```
import cv2

cap = cv2.VideoCapture(0)

while True:
```

```
    ret, frame = cap.read()
    if not ret: break

    cv2.imshow('Camera', frame)

    key = cv2.waitKey(1) & 0xFF     # & 0xFF = safe way to read key

    if key == ord('q'):             # 'q' = quit
        print('Quit!')
        break
    elif key == ord('s'):           # 's' = screenshot save karo
        cv2.imwrite('screenshot.jpg', frame)
        print('Screenshot saved!')
    elif key == ord('g'):           # 'g' = grayscale toggle
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        cv2.imshow('Gray', gray)

cap.release()
cv2.destroyAllWindows()
```

& 0xFF kyun likhte hain?

Kuch systems mein waitKey() bada number return karta hai.

& 0xFF se hum sirf last 8 bits lete hain — jo actual key code hota hai.

Yeh ek safe practice hai — hamesha aise hi likhna.

Try it yourself:

1. Live loop mein 's' key se screenshot save karo.
2. Folder mein jaao — screenshot.jpg bana?
3. 'q' se band karo.

Mini Project: Live Traffic Monitor — Webcam pe Drawing + Text

Sab topics ek saath — live webcam, rectangle, text overlay, aur keyboard controls:

```
# Day 4 Mini Project — Live Traffic Monitor
import cv2
```

```
cap = cv2.VideoCapture(0)

# Simulated lane counts (baad mein YOLO se aayenge!)
lane_counts = {'North': 12, 'South': 5, 'East': 20, 'West': 8}
frame_number = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_number += 1

    # === Drawing zone ===

    # Title bar — top pe blue rectangle
    cv2.rectangle(frame, (0, 0), (640, 40), (180, 95, 24), -1)  # -1=filled
    cv2.putText(frame, 'SMART TRAFFIC MONITOR', (10, 28),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

    # Lane counts dikhao
    for i, (lane, count) in enumerate(lane_counts.items()):
        y = 80 + i * 35
        color = (0, 255, 0) if count > 10 else (0, 200, 255)
        cv2.putText(frame, f'{lane}: {count} vehicles',
                    (10, y), cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

    # Frame counter — bottom right
    cv2.putText(frame, f'Frame: {frame_number}', (480, 460),
                cv2.FONT_HERSHEY_SIMPLEX, 0.5, (150, 150, 150), 1)

    # Busiest lane highlight karo
    busiest = max(lane_counts, key=lane_counts.get)
    cv2.putText(frame, f'Busiest: {busiest}!', (10, 240),
```

```
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

    cv2.imshow('Traffic Monitor', frame)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break
    elif key == ord('s'):
        cv2.imwrite('monitor_screenshot.jpg', frame)
        print('Screenshot saved!')

cap.release()
cv2.destroyAllWindows()
```

Yeh kahan kaam aayega?

Day 5 mein: lane_counts real YOLO detection se aayenge — yeh code waise ka waisa kaam karega!

Day 6-7 mein: 4 cameras ke liye 4 alag VideoCapture objects honge.

Abhi counts hardcoded hain — random.randint(0, 25) se counts badlo aur dekho live update.

Ghar pe challenge (optional)

- lane_counts ek dictionary hai — 'q' dabane se pehle ek lane ka count badlao mid-loop. Hint: lane_counts['North'] += 1
- Advanced: 's' key se screenshot aur uska frame number bhi filename mein daalo. Hint: cv2.imwrite(f'frame_{frame_number}.jpg', frame)

Day 5

Smart Traffic System Workshop

YOLO Introduction + Object Detection

Ultralytics install | Model load | Image detection | Results padhna | Live webcam | Mini project

Day recap

- cv2.VideoCapture(0) — webcam se live feed lena
- cap.read() — ek frame lena loop mein
- cv2.rectangle(), cv2.circle(), cv2.line() — shapes draw karna
- cv2.putText() — text overlay karna frame pe
- cv2.waitKey(1) & 0xFF + ord('q') — keyboard controls

Topic 1 — YOLO kya hai?

Real life analogy

Sochlo ek bahut trained security guard hai. Tum usse ek bheed ki photo dikhao — woh ek second mein

bata dega: "yahan ek car hai, wahan ek truck hai, us corner mein ek motorbike hai."

Aur woh exactly yeh bhi batata hai ki kahan hai — ek box kheench ke.

YOLO = You Only Look Once.

Normal detection systems image ko kai baar scan karte hain.

YOLO ek baar mein poori image dekhta hai — isliye bahut fast hai.

Image → YOLO Model → Results:

- Class (car, truck, person...)

- Confidence (kitna sure hai — 0.0 to 1.0)

- Bounding Box (x1, y1, x2, y2)

Traffic system mein: YOLO har frame mein cars, trucks, bikes detect karega —

phir hum count karenge aur signal timing decide karenge.

YOLO detection example — tumhare traffic image pe:

Topic 2 — Ultralytics Install aur Model Load karna

Install karo — terminal mein ek baar:

```
pip install ultralytics
```

> **Note:** Yeh thoda time le sakta hai — pehli baar 100MB+ download hota hai.
>
> Internet connection acchi honi chahiye.

Import aur version check karo:

```
from ultralytics import YOLO

import cv2


# Version check karo

import ultralytics

print('Ultralytics version:', ultralytics.__version__)
```

Output / Terminal:

```
Ultralytics version: 8.x.x
```

Model load karna:

```
from ultralytics import YOLO


# YOLOv8n load karo — 'n' = nano (sabse chhota aur fast)

model = YOLO('yolov8n.pt')


print('Model loaded!')

print('Classes YOLO detect kar sakta hai:', len(model.names))

print('Pehli 10 classes:', list(model.names.values())[:10])
```

Output / Terminal:

```
Model loaded!

Classes YOLO detect kar sakta hai: 80

Pehli 10 classes: ['person', 'bicycle', 'car', 'motorcycle', 'airplane',

                   'bus', 'train', 'truck', 'boat', 'traffic light']
```

yolov8n.pt kya hai?

Pehli baar run karne pe yeh file automatically download hoti hai internet se.

.pt = PyTorch model file — YOLO ka "brain" hai yahi.

n = nano — 4 versions hain: n (fastest), s, m, l (most accurate).

Hum traffic system ke liye yolov8n.pt use karenge — speed important hai!

Model	Size	Speed	Accuracy	Traffic system ke liye
`yolov8n.pt`	~6MB	Bahut fast	Theek hai	Hum yahi use karenge
`yolov8s.pt`	~22MB	Fast	Achhi	Advanced use
`yolov8m.pt`	~52MB	Medium	Bahut achhi	High-end hardware
`yolov8l.pt`	~87MB	Slow	Best	Research

Try it yourself:

1. Install karo aur version print karo.
2. Model load karo — download hua?
3. model.names print karo — kaunsi classes hain?

Topic 3 — Image pe Detection Chalana

Real life analogy

Pehle hum YOLO ko ek photo dikhate hain — bilkul jaise tum kisi expert ko ek traffic junction ki photo dikhao aur poocho "ismein kya kya dikh raha hai?"

model(image) = yahi kaam karta hai — model ko image do, woh results return karta hai.

Basic detection code:

```
from ultralytics import YOLO

import cv2


# Model load karo

model = YOLO('yolov8n.pt')


# Image load karo
```

```
img = cv2.imread('traffic.png')


# Detection chalao!

results = model(img)


print('Detection complete!')

print('Type of results:', type(results))

print('Kitne results:', len(results))
```

Output / Terminal:

```
Detection complete!

Type of results: <class 'list'>

Kitne results: 1
```

results ek list kyun hai?

YOLO ek baar mein multiple images process kar sakta hai.

Hum ek image dete hain toh list mein ek result aata hai.

results[0] se pehla (aur akela) result milta hai.

Try it yourself:

1. Model load karo aur apni traffic.png pe detection chalao.
2. results print karo — kya dikhta hai?

Topic 4 — Results Padhna (Boxes, Classes, Confidence)

Real life analogy

YOLO ne photo dekhi — ab woh tumhe report deta hai:

"Maine 3 cheezein dekhi — ek car yahan hai, ek truck wahan hai, ek motorbike corner mein."

Har cheez ke saath batata hai: class name, confidence score, aur bounding box coordinates.

Results ko detail mein padhna:

```
from ultralytics import YOLO

import cv2


model = YOLO('yolov8n.pt')

img = cv2.imread('traffic.png')

results = model(img)
```

```
# Pehle result lo

result = results[0]

boxes  = result.boxes


print(f'Total detections: {len(boxes)}')

print('---')


# Har detection ke baare mein print karo

for i, box in enumerate(boxes):

    class_id   = int(box.cls[0])

    class_name = model.names[class_id]

    confidence = float(box.conf[0])

    x1, y1, x2, y2 = box.xyxy[0]

    x1, y1, x2, y2 = int(x1), int(y1), int(x2), int(y2)


    print(f'Detection {i+1}:')

    print(f'  Class     : {class_name} (ID: {class_id})')

    print(f'  Confidence: {confidence:.2f}')

    print(f'  Box       : ({x1}, {y1}) to ({x2}, {y2})')

    print('---')
```

Output / Terminal:

```
Total detections: 3

---

Detection 1:

  Class     : car (ID: 2)

  Confidence: 0.87

  Box       : (45, 120) to (280, 310)

---

Detection 2:

  Class     : truck (ID: 7)

  Confidence: 0.74

  Box       : (310, 90) to (580, 350)

---
```

Property	Kya milta hai?	Example
`box.cls[0]`	Class ID — number	2 (= car)
`model.names[class_id]`	Class name — string	'car'
`box.conf[0]`	Confidence — 0.0 to 1.0	0.87
`box.xyxy[0]`	Box coordinates (x1,y1,x2,y2)	(45,120,280,310)

Try it yourself:

1. Apni image pe detection chalao.
2. Har detection ka class name aur confidence print karo.
3. Confidence 0.5 se kam hai toh skip karo — if confidence < 0.5: continue

Topic 5 — Bounding Boxes Draw karna Detected Objects pe

Real life analogy

YOLO ne bataya ki car kahan hai — ab hum us car ke around ek green box kheenchte hain, aur uska naam aur confidence bhi likhte hain.

Yahi hum live CCTV mein dekhte hain!

Yahan Day 4 ka drawing knowledge kaam aata hai — cv2.rectangle() + cv2.putText().

Boxes draw karna — poora code:

```
from ultralytics import YOLO

import cv2


model = YOLO('yolov8n.pt')

img = cv2.imread('traffic.png')

results = model(img)

result  = results[0]


# Detected image pe boxes draw karo

for box in result.boxes:

    class_id   = int(box.cls[0])

    class_name = model.names[class_id]

    confidence = float(box.conf[0])


    if confidence < 0.5:        # Low confidence skip karo

        continue
```

```
    x1, y1, x2, y2 = map(int, box.xyxy[0])

    # Rectangle draw karo — green box
    cv2.rectangle(img, (x1, y1), (x2, y2), (0, 255, 0), 2)

    # Label banao: "car 0.87"
    label = f'{class_name} {confidence:.2f}'

    # Label ke peeche background rectangle
    (lw, lh), _ = cv2.getTextSize(
        label, cv2.FONT_HERSHEY_SIMPLEX, 0.6, 2)
    cv2.rectangle(img,
                  (x1, y1 - lh - 8), (x1 + lw, y1),
                  (0, 255, 0), -1)                # Filled green

    cv2.putText(img, label, (x1, y1 - 5),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6,
                (0, 0, 0), 2)                     # Black text

# ■ Puri image screen pe dikhao — fit hogi!
cv2.namedWindow('YOLO Detection', cv2.WINDOW_NORMAL)
cv2.resizeWindow('YOLO Detection', 1280, 720)
cv2.imshow('YOLO Detection', img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

cv2.getTextSize() kyun use kiya?

Text ke peeche background rectangle draw karne ke liye pehle text ki width/height jaanni padti hai.

getTextSize() exactly wahi batata hai — toh hum perfectly fitting background bana sakte hain.

cv2.WINDOW_NORMAL + resizeWindow() kyun?

Camera ya badi image window screen se badi ho sakti hai — clip ho jaati hai.

WINDOW_NORMAL se window manually resize ho sakti hai, resizeWindow() se fixed size set hoti hai.

Try it yourself:

1. Code chalao — boxes dikh rahe hain?
2. Label ka color badlo — green se red karo.
3. Confidence threshold 0.5 se 0.7 karo — kam boxes dikhenge?

Mini Project: Live YOLO Traffic Detector

Sab topics ek saath — live webcam feed pe real-time YOLO detection, bounding boxes, aur vehicle count:

```
# Day 5 Mini Project — Live YOLO Traffic Detector

from ultralytics import YOLO

import cv2


model = YOLO('yolov8n.pt')


# Traffic system ke liye relevant classes

VEHICLE_CLASSES = ['car', 'truck', 'bus', 'motorcycle', 'bicycle']


cap = cv2.VideoCapture(0)

print('Camera ready — q dabao band karne ke liye')


while True:

    ret, frame = cap.read()

    if not ret: break


    # YOLO detection chalao

    results = model(frame, verbose=False)  # verbose=False = no spam

    result  = results[0]


    vehicle_count = 0


    for box in result.boxes:

        class_id   = int(box.cls[0])

        class_name = model.names[class_id]

        confidence = float(box.conf[0])


        if class_name not in VEHICLE_CLASSES: continue
```

```
        if confidence < 0.5: continue

        vehicle_count += 1
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)

        label = f'{class_name} {confidence:.2f}'
        cv2.putText(frame, label, (x1, y1 - 8),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.55, (0, 255, 0), 2)

    # Title bar
    cv2.rectangle(frame, (0, 0), (640, 45), (30, 30, 30), -1)
    cv2.putText(frame, 'YOLO TRAFFIC DETECTOR', (10, 30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.75, (255, 255, 255), 2)

    # Vehicle count
    count_color = (0,255,0) if vehicle_count <= 5 else (0,0,255)
    cv2.putText(frame, f'Vehicles: {vehicle_count}', (10, 85),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, count_color, 2)

    cv2.imshow('Live Traffic Detector', frame)

    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'): break
    elif key == ord('s'):
        cv2.imwrite('detection_screenshot.jpg', frame)
        print(f'Screenshot saved! Vehicles: {vehicle_count}')

cap.release()
cv2.destroyAllWindows()
```

Yeh kahan kaam aayega?

Day 6 mein: vehicle_count ko lane-wise track karenge — North, South, East, West alag alag.

Day 7 mein: is count ke basis pe signal timing automatically adjust hogi.

Yahi core logic hai jo poore Smart Traffic System ka dil hai!

Ghar pe challenge (optional)

- VEHICLE_CLASSES list mein se 'bicycle' hata do — count change hota hai?
- Confidence threshold 0.5 se 0.3 karo — zyada detections aate hain? Galat bhi aate hain?
- Advanced: frame pe alag alag vehicles ke liye alag color — car=green, truck=red, bike=yellow. Hint: CLASS_COLORS = {'car': (0,255,0), 'truck': (0,0,255), ...}